```
import os
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
```

```
import torch
torch.cuda.empty_cache()
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cuda
```

```
import os
!pip install datasets

from datasets import load_dataset

algebra_ds = load_dataset("EleutherAI/hendrycks_math", "algebra")
geometry_ds = load_dataset("EleutherAI/hendrycks_math", "geometry")
number_theory_ds = load_dataset("EleutherAI/hendrycks_math", "number_theory")
precalculus_ds = load_dataset("EleutherAI/hendrycks_math", "precalculus")

subjects = [
    "algebra",
    "counting_and_probability",
    "geometry",
    "intermediate_algebra",
    "number_theory",
    "prealgebra",
    "precalculus"
]

datasets_list = []

for subject in subjects:
    ds = load_dataset("EleutherAI/hendrycks_math", subject)
    datasets_list.append(ds["train"])

from datasets import concatenate_datasets

combined_ds = concatenate_datasets(datasets_list)
combined_ds = combined_ds.add_column("original_index", list(range(len(combined_ds))))

# Ensure the directory exists before saving and mounting drive later
# os.makedirs("/content/drive/MyDrive/math_datasets", exist_ok=True)

from google.colab import drive
drive.mount('/content/drive', force_remount=True)

combined_ds.save_to_disk("/content/drive/MyDrive/math_datasets/hendrycks_math_full")
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.8.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.29.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (24.0.0)
Requirement already satisfied: dill<0.4.2,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: httpx<1.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.28.1)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.20 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2026.2.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=20
Requirement already satisfied: huggingface-hub<2.0,>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.11
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=202
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (4.13.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (2026.4.22)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (3.13)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0->datase
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.25.0->datasets
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->d
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->dataset
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2026.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1-
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fs
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1-
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->dat
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=0.
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=0.
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->h
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
Mounted at /content/drive
```

Saving the dataset (1/1 shards): 100% 7500/7500 [00:00<00:00, 19986.08 examples/s]

```
print(combined_ds)
print(combined_ds[0])
```

```
Dataset({
    features: ['problem', 'level', 'type', 'solution', 'original_index'],
    num_rows: 7500
})
{'problem': 'Let \\[f(x) = \\left\\{\n\\begin{array}{cl} ax+3, &\\text{ if }x>2, \\\\\nx-5 &\\text{ if } -2 \\le x \\le 2, \
```

PREPROCESSING

```
def preprocess(example):
    example["text"] = f"Problem:\n{example['problem']}\n\nSolution:\n{example['solution']}"
    return example

processed_ds = combined_ds.map(preprocess)
```

```
# imo_ds = load_dataset("TamasSimonds/imo-dataset")
```

```
# def preprocess_imo(example):
#     example["text"] = f"Problem:\n{example['prompt']}\n\nSolution:\n{example['completion']}"
#     return example

# imo_processed = imo_ds["train"].map(preprocess_imo)
```

```
processed_ds.save_to_disk("/content/drive/MyDrive/math_datasets/hendrycks_processed")
# imo_processed.save_to_disk("/content/drive/MyDrive/math_datasets/imo_processed")
```

```
Saving the dataset (1/1 shards): 100%  7500/7500 [00:00<00:00, 76637.59 examples/s]
```

```
split_ds = processed_ds.train_test_split(test_size=0.1)
train_ds = split_ds["train"]
test_ds = split_ds["test"]
```

LOAD MODEL

```
!pip install transformers peft accelerate bitsandbytes trl
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: peft in /usr/local/lib/python3.12/dist-packages (0.19.1)
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.13.0)
Collecting bitsandbytes
  Downloading bitsandbytes-0.49.2-py3-none-manylinux_2_24_x86_64.whl.metadata (10 kB)
Collecting trl
  Downloading trl-1.3.0-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.29.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from peft) (5.9.5)
Requirement already satisfied: torch>=1.13.0 in /usr/local/lib/python3.12/dist-packages (from peft) (2.10.0+cu128)
Collecting datasets>=4.7.0 (from trl)
  Downloading datasets-4.8.5-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from trl) (3.1.6)
Collecting pyarrow>=21.0.0 (from datasets>=4.7.0->trl)
  Downloading pyarrow-24.0.0-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (3.0 kB)
Requirement already satisfied: dill<0.4.2,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets>=4.7.0->trl) (
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets>=4.7.0->trl) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets>=4.7.0->trl) (2.
Requirement already satisfied: httpx<1.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets>=4.7.0->trl) (0.28.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets>=4.7.0->trl) (3.6.0)
Requirement already satisfied: multiprocess<0.70.20 in /usr/local/lib/python3.12/dist-packages (from datasets>=4.7.0->trl)
Requirement already satisfied: fsspec<=2026.2.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transfo
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->peft) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->peft) (1.14.0
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->peft) (3.6.
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->peft)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.1
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.1
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.1
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.1
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->pe
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->p
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->peft) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->trl) (3.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets>=4.7.0->trl) (
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
import torch

model_name = "Qwen/Qwen2.5-Math-1.5B"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4"
)

tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)
```

```
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
Requirement al                                                    ) in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->type
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12
Downloading bitsandb                                                inux_2_24_x86_64.whl (60.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.7/60.7 MB 11.7 MB/s eta 0:00:00
Downloadin                                                          721 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 721.6/721.6 kB 32.4 MB/s eta 0:00:00
Downloadin                                                          (528 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 529.0/529.0 kB 41.8 MB/s eta 0:00:00
Downloading pyarrow-24.0.0-cp312-cp312-manylinux_2_28_x86_64.whl (48.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48.9/48.9 MB 14.2 MB/s eta 0:00:00
Installing collected packages: pyarrow, bitsandbytes, datasets, trl
  Attempting uninst
    Found existing installation: pyarrow 18.1.0
    Uninstalling pyarrow-
      Successfully uninstalled pyarrow-18.1.0
  Attempting uninstall: datasets
```

config.json: 100% 676/676 [00:00<00:00, 25.6kB/s]
tokenizer_config.json: 7.30k/? [00:00<00:00, 124kB/s]
vocab.json: 2.78M/? [00:00<00:00, 27.3MB/s]
merges.txt: 1.67M/? [00:00<00:00, 42.8MB/s]
tokenizer.json: 7.03M/? [00:00<00:00, 52.4MB/s]
model.safetensors: 100% 3.09G/3.09G [00:38<00:00, 61.4MB/s]
Loading weights: 100% 338/338 [00:10<00:00, 291.79it/s, Materializing param=model.norm.weight]
generation_config.json: 100% 138/138 [00:00<00:00, 6.30kB/s]

```
sample = test_ds[0]
problem = sample["problem"]

prompt = f"""Solve this olympiad math problem step by step.

Problem:
{problem}

Clearly explain your reasoning and state the final answer at the end.
"""
```

WITHOUT FINE TUNING

```
import re
import json
import torch
from datasets import load_from_disk
import os # Added import for os module

# ====================================================
# CREATE FIXED HENDRYCKS ALGEBRA BENCHMARK (if it doesn't exist)
# ====================================================
fixed_benchmark_path = "/content/drive/MyDrive/math_datasets/fixed_algebra_500"

if not os.path.exists(fixed_benchmark_path):
    print(f"Fixed algebra benchmark not found at {fixed_benchmark_path}. Creating it now...")
    # Load the processed dataset (assuming 'hendrycks_processed' exists from prior steps)
    loaded_processed_ds = load_from_disk("/content/drive/MyDrive/math_datasets/hendrycks_processed")

    # Filter for algebra problems
    algebra_ds_filtered = loaded_processed_ds.filter(lambda x: x["type"] == "Algebra")

    # Take the first 500 samples for a fixed benchmark
    # Ensure not to exceed available samples if less than 500
    fixed_algebra_benchmark = algebra_ds_filtered.select(range(min(500, len(algebra_ds_filtered))))

    # Save the fixed benchmark
    fixed_algebra_benchmark.save_to_disk(fixed_benchmark_path)
    print(f"Created fixed algebra benchmark with {len(fixed_algebra_benchmark)} questions.")
else:
    print(f"Fixed algebra benchmark found at {fixed_benchmark_path}. Loading existing dataset.")


# ====================================================
# LOAD FIXED HENDRYCKS ALGEBRA BENCHMARK
# ====================================================

fixed_algebra_test = load_from_disk(
    fixed_benchmark_path
)

print("Loaded fixed benchmark questions:", len(fixed_algebra_test))


# ====================================================
# ROBUST ANSWER EXTRACTION
# ====================================================

def extract_boxed_answer(text):
```

```
    # Try boxed answer first
    boxed = re.findall(r'\\boxed\{(.*?)\}', text)

    if boxed:
        return boxed[-1].strip()

    # Fallback to last number
    fallback = re.findall(r'[-+]?\d*\.?\d+', text)

    if fallback:
        return fallback[-1]

    # If no valid answer found
    return None


# ====================================================
# EVALUATION SETTINGS
# ====================================================

correct = 0
total_questions = len(fixed_algebra_test)

results = []


# ====================================================
# MAIN LOOP
# ====================================================

for i in range(total_questions):

    sample = fixed_algebra_test[i]
    problem = sample["problem"]

    prompt = f"""Solve this olympiad algebra problem step by step.

Problem:
{problem}

Clearly explain every reasoning step and state the final answer in \boxed{{}} format.
"""

    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    ).to(device)

    with torch.no_grad():

        outputs = model.generate(
            **inputs,
            max_new_tokens=256,
            do_sample=False
        )

    response = tokenizer.decode(
        outputs[0],
        skip_special_tokens=True
    )

    predicted_answer = extract_boxed_answer(response)
    actual_answer = extract_boxed_answer(sample["solution"])


    # ====================================================
    # STRICT ACCURACY LOGIC
    # ====================================================

    # If predicted answer is missing, auto-fail
    if predicted_answer is None:
        is_correct = False

    # If actual answer missing, also fail
    elif actual_answer is None:
        is_correct = False

    # Normal comparison
    else:
```

```
            is_correct = predicted_answer == actual_answer


        if is_correct:
            correct += 1


        # ====================================================
        # SAVE RESULTS
        # ====================================================

        results.append({
            "question_id": i,
            "original_dataset_index": sample["original_index"],
            "category": sample["type"],
            "difficulty": sample["level"],
            "problem": sample["problem"],
            "generated_solution": response,
            "actual_solution": sample["solution"],
            "predicted_answer": predicted_answer if predicted_answer is not None else "NO ANSWER",
            "actual_answer": actual_answer if actual_answer is not None else "UNKNOWN",
            "correct": is_correct
        })


        # ====================================================
        # DISPLAY RESULTS
        # ====================================================

        print("\n" + "=" * 120)
        print(f"QUESTION {i+1}/{total_questions}")
        print(f"Original Dataset Index : {sample['original_index']}")
        print(f"Category               : {sample['type']}")
        print(f"Difficulty Level       : {sample['level']}")
        print("=" * 120)

        print("\nALGEBRA PROBLEM:\n")
        print(sample["problem"])

        print("\nMODEL STEP-BY-STEP GENERATED SOLUTION:\n")
        print(response)

        print("\nACTUAL HENDRYCKS DATASET SOLUTION:\n")
        print(sample["solution"])

        print("\nFINAL ANSWER COMPARISON:")
        print(f"Predicted Answer       : {predicted_answer if predicted_answer is not None else 'NO ANSWER'}")
        print(f"Actual Answer          : {actual_answer if actual_answer is not None else 'UNKNOWN'}")
        print(f"Correct                : {'YES' if is_correct else 'NO'}")

        print("=" * 120 + "\n")


    # ====================================================
    # FINAL PERFORMANCE SUMMARY
    # ====================================================

    accuracy = (correct / total_questions) * 100

    print("\nFINAL PERFORMANCE SUMMARY")
    print("=" * 70)
    print(f"Dataset Used           : Hendrycks MATH (Algebra Only)")
    print(f"Total Questions Tested : {total_questions}")
    print(f"Correct Answers        : {correct}")
    print(f"Incorrect Answers      : {total_questions - correct}")
    print(f"Final Accuracy         : {accuracy:.2f}%")
    print("=" * 70)


    # ====================================================
    # SAVE DETAILED RESULTS
    # ====================================================

    with open(
        "/content/drive/MyDrive/math_datasets/baseline_results_detailed.json",
        "w"
    ) as f:
        json.dump(results, f, indent=4)

    print("Detailed results saved successfully.")
```

```
Fixed algebra benchmark not found at /content/drive/MyDrive/math_datasets/fixed_algebra_500. Creating it now...
```

Filter: 100% 7500/7500 [00:00<00:00, 30835.94 examples/s]

Saving the dataset (1/1 shards): 100% 500/500 [00:00<00:00, 9350.31 examples/s]

```
Setting `pad_token_id` to `eos_token_id`:151643 for open-end generation.
Created fixed algebra benchmark with 500 questions.
Loaded fixed benchmark questions: 500
Setting `pad_token_id` to `eos_token_id`:151643 for open-end generation.

================================================================================================================================================
QUESTION 1/500
Original Dataset Index : 0
Category               : Algebra
Difficulty Level       : Level 5
================================================================================================================================================

ALGEBRA PROBLEM:

Let \[f(x) = \left\{
\begin{array}{cl} ax+3, &\text{ if }x>2, \\
x-5 &\text{ if } -2 \le x \le 2, \\
2x-b &\text{ if } x <-2.
\end{array}
\right.\]Find $a+b$ if the piecewise function is continuous (which means that its graph can be drawn without lifting your

MODEL STEP-BY-STEP GENERATED SOLUTION:

Solve this olympiad algebra problem step by step.

Problem:
Let \[f(x) = \left\{
\begin{array}{cl} ax+3, &\text{ if }x>2, \\
x-5 &\text{ if } -2 \le x \le 2, \\
2x-b &\text{ if } x <-2.
\end{array}
\right.\]Find $a+b$ if the piecewise function is continuous (which means that its graph can be drawn without lifting your

Clearly explain every reasoning step and state the final answer inoxed{} format.
To solve the problem, we need to ensure that the piecewise function \( f(x) \) is continuous. This means that the left-han

Let's break it down step-by-step:

1. **Continuity at \( x = 2 \):**
   - The left-hand limit as \( x \) approaches 2 is given by the function \( x - 5 \).
   - The right-hand limit as \( x \) approaches 2 is given by the function \( ax + 3 \).
   - The value of the function at \( x = 2 \) is \( 2 - 5 = -3 \).
   - So, we need \( \lim_{x \to 2^-} f(x) = \lim_{x \to 2^+} f(x) = f(2) \).
   - This gives us the equation: \( 2 - 5 = 2a + 3 \).

2. **Continuity at \( x =

ACTUAL HENDRYCKS DATASET SOLUTION:

For the piecewise function to be continuous, the cases must "meet" at $2$ and $-2$. For example, $ax+3$ and $x-5$ must be

FINAL ANSWER COMPARISON:
Predicted Answer       : 2
```

```
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)
```

Loading weights: 100% 338/338 [00:02<00:00, 399.75it/s, Materializing param=model.norm.weight]

```
================================================================================================================================================
QUESTION 2/500
Original Dataset Index : 1
Category               : Algebra
Difficulty Level       : Level 5
================================================================================================================================================
```

FINE TUNING

```
from peft import LoraConfig, get_peft_model
lora_config = LoraConfig(
r=16,
lora_alpha=32,
target_modules=["q_proj", "v_proj"],
lora_dropout=0.05,
bias="none",
task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)

model.print_trainable_parameters()
```

trainable params: 2,179,072 || all params: 1,545,803,776 || trainable%: 0.1410

TOKENIZATION

```
We can iterate over possible values of \( m \) and \( r \) to find the largest \( n \) that satisfies all these conditions
```

```
def tokenize_function(example):
    tokens = tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=512
    )

    tokens["labels"] = tokens["input_ids"].copy()

    return tokens
```

```
train_tokenized = train_ds.map(tokenize_function, batched=False)
test_tokenized = test_ds.map(tokenize_function, batched=False)
```

```
Map: 100%                                                 6750/6750 [00:09<00:00, 725.43 examples/s]
Map: 100%                                                 750/750 [00:01<00:00, 993.45 examples/s]
```

TRAIN THE MODEL

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="/content/drive/MyDrive/qwen_math_finetuned",
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=4,
    num_train_epochs=2,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_steps=50,
    learning_rate=2e-5,
    fp16=True,
    report_to="none",
    remove_unused_columns=False
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tokenized,
    eval_dataset=test_tokenized
)

trainer.train()
```

[3376/3376 1:46:09, Epoch 2/2]

Epoch	Training Loss	Validation Loss
1	0.268108	0.263881
2	0.279160	0.260497

```
TrainOutput(global_step=3376, training_loss=0.3405177485886343, metrics={'train_runtime': 6371.6641,
'train_samples_per_second': 2.119, 'train_steps_per_second': 0.53, 'total_flos': 5.443281616896e+16, 'train_loss':
```

```
trainer.save_model("/content/drive/MyDrive/qwen_math_final")
tokenizer.save_pretrained("/content/drive/MyDrive/qwen_math_final")
```

```
('/content/drive/MyDrive/qwen_math_final/tokenizer_config.json',
 '/content/drive/MyDrive/qwen_math_final/chat_template.jinja',
 '/content/drive/MyDrive/qwen_math_final/tokenizer.json')
```

```
import re
import json
import torch
from datasets import load_from_disk
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
from peft import PeftModel
import os

# ====================================================
# DEVICE SETUP
# ====================================================

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
# ====================================================
# LOAD BASE MODEL + FINE-TUNED LORA MODEL
# ====================================================

model_name = "Qwen/Qwen2.5-Math-1.5B"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4"
)

tokenizer = AutoTokenizer.from_pretrained(model_name)

# Load base quantized model
base_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)

# Load your fine-tuned LoRA checkpoint
model = PeftModel.from_pretrained(
    base_model,
    "/content/drive/MyDrive/qwen_math_final"
)

model.eval()

# ====================================================
# CREATE FIXED HENDRYCKS ALGEBRA BENCHMARK (if not exist)
# ====================================================

fixed_benchmark_path = "/content/drive/MyDrive/math_datasets/fixed_algebra_500"

if not os.path.exists(fixed_benchmark_path):
    print(f"Fixed algebra benchmark not found at {fixed_benchmark_path}. Creating it now...")

    loaded_processed_ds = load_from_disk(
        "/content/drive/MyDrive/math_datasets/hendrycks_processed"
    )

    algebra_ds_filtered = loaded_processed_ds.filter(
        lambda x: x["type"] == "Algebra"
    )

    fixed_algebra_benchmark = algebra_ds_filtered.select(
        range(min(500, len(algebra_ds_filtered)))
    )

    fixed_algebra_benchmark.save_to_disk(fixed_benchmark_path)

    print(
        f"Created fixed algebra benchmark with {len(fixed_algebra_benchmark)} questions."
    )

else:
    print(
        f"Fixed algebra benchmark found at {fixed_benchmark_path}. Loading existing dataset."
    )

# ====================================================
# LOAD FIXED BENCHMARK
# ====================================================

fixed_algebra_test = load_from_disk(
    fixed_benchmark_path
)

print("Loaded fixed benchmark questions:", len(fixed_algebra_test))

# ====================================================
# ROBUST ANSWER EXTRACTION
# ====================================================

def extract_boxed_answer(text):

    boxed = re.findall(r'\\boxed\{(.*?)\}', text)

    if boxed:
        return boxed[-1].strip()
```

```
    fallback = re.findall(r'[-+]?\d*\.?\d+', text)

    if fallback:
        return fallback[-1]

    return None


# ====================================================
# EVALUATION SETTINGS
# ====================================================

correct = 0
total_questions = len(fixed_algebra_test)

results = []

# ====================================================
# MAIN LOOP
# ====================================================

for i in range(total_questions):

    sample = fixed_algebra_test[i]
    problem = sample["problem"]

    prompt = f"""Solve this olympiad algebra problem step by step.

Problem:
{problem}

Clearly explain every reasoning step and state the final answer in \\boxed{{}} format.
"""

    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    ).to(device)

    with torch.no_grad():

        outputs = model.generate(
            **inputs,
            max_new_tokens=256,
            do_sample=False
        )

    response = tokenizer.decode(
        outputs[0],
        skip_special_tokens=True
    )

    predicted_answer = extract_boxed_answer(response)
    actual_answer = extract_boxed_answer(sample["solution"])

    # ====================================================
    # STRICT ACCURACY LOGIC
    # ====================================================

    if predicted_answer is None:
        is_correct = False
    elif actual_answer is None:
        is_correct = False
    else:
        is_correct = predicted_answer == actual_answer

    if is_correct:
        correct += 1

    # ====================================================
    # SAVE RESULTS
    # ====================================================

    results.append({
        "question_id": i,
        "original_dataset_index": sample["original_index"],
        "category": sample["type"],
        "difficulty": sample["level"],
        "problem": sample["problem"],
```

```
            "generated_solution": response,
            "actual_solution": sample["solution"],
            "predicted_answer": predicted_answer if predicted_answer is not None else "NO ANSWER",
            "actual_answer": actual_answer if actual_answer is not None else "UNKNOWN",
            "correct": is_correct
        })

        # ==================================================
        # DISPLAY RESULTS
        # ==================================================

        print("\n" + "=" * 120)
        print(f"QUESTION {i+1}/{total_questions}")
        print(f"Original Dataset Index : {sample['original_index']}")
        print(f"Category               : {sample['type']}")
        print(f"Difficulty Level       : {sample['level']}")
        print("=" * 120)

        print("\nALGEBRA PROBLEM:\n")
        print(sample["problem"])

        print("\nFINE-TUNED MODEL STEP-BY-STEP GENERATED SOLUTION:\n")
        print(response)

        print("\nACTUAL HENDRYCKS DATASET SOLUTION:\n")
        print(sample["solution"])

        print("\nFINAL ANSWER COMPARISON:")
        print(
            f"Predicted Answer       : {predicted_answer if predicted_answer is not None else 'NO ANSWER'}"
        )
        print(
            f"Actual Answer          : {actual_answer if actual_answer is not None else 'UNKNOWN'}"
        )
        print(f"Correct                : {'YES' if is_correct else 'NO'}")

        print("=" * 120 + "\n")

    # ==================================================
    # FINAL PERFORMANCE SUMMARY
    # ==================================================

    accuracy = (correct / total_questions) * 100

    print("\nFINAL PERFORMANCE SUMMARY")
    print("=" * 70)
    print(f"Dataset Used           : Hendrycks MATH (Algebra Only)")
    print(f"Model Type             : Fine-Tuned Qwen2.5-Math + LoRA")
    print(f"Total Questions Tested : {total_questions}")
    print(f"Correct Answers        : {correct}")
    print(f"Incorrect Answers      : {total_questions - correct}")
    print(f"Final Accuracy         : {accuracy:.2f}%")
    print("=" * 70)

    # ==================================================
    # SAVE DETAILED RESULTS
    # ==================================================

    with open(
        "/content/drive/MyDrive/math_datasets/fine_tuned_results_detailed.json",
        "w"
    ) as f:
        json.dump(results, f, indent=4)

    print("Fine-tuned detailed results saved successfully.")
```

```
Setting `pad_token_id` to `eos_token_id`:151643 for open-end generation.

====================================================================================================================================================
QUESTION 9/500
Original Dataset Index : 8
Category               : Algebra
Difficulty Level       : Level 2
====================================================================================================================================================

ALGEBRA PROBLEM:

The sequence of integers in the row of squares and in each of the two columns of squares form three distinct arithmetic se

[asy]
unitsize(0.35inch);
draw((0,0)--(7,0)--(7,1)--(0,1)--cycle);
draw((1,0)--(1,1));
```

```
draw((2,0)--(2,1));
Using device: cuda
draw((3,0)--(3,1));
Loading weights: 100%|██████████| 338/338 [00:01<00:00, 382.38it/s, Materializing param=model.norm.weight]
draw((4,0)--(4,1));
draw((5,0)--(5,1));
Setting `pad_token_id` to `eos_token_id`:151643 for open-end generation.
draw((6,0)--(6,1));
Fixed algebra benchmark found at /content/drive/MyDrive/math_datasets/fixed_algebra_500. Loading existing dataset.
draw((6,2)--(7,2)--(7,-4)--(6,-4)--cycle);
Loaded fixed benchmark questions: 500
draw((6,-1)--(7,-1));
Setting `pad_token_id` to `eos_token_id`:151643 for open-end generation.
draw((6,-2)--(7,-2));
draw((6,-3)--(7,-3));
================================================================================================================
draw((3,0)--(4,0)--(4,-3)--(3,-3)--cycle);
QUESTION 1/500
draw((3,-1)--(4,-1));
Original Dataset Index : 0
draw((3,-2)--(4,-2));
Category : Algebra
label("21",(0.5,0.8),S);
Difficulty Level : Level 5
label("14",(3.5,-1.2),S);
================================================================================================================
label("18",(3.5,-2.2),S);
label("$N$",(6.5,1.8),S);
ALGEBRA PROBLEM:
label("-17",(6.5,-3.2),S);
[/asy]
Let \[f(x) = \left\{
MODEL STEP-BY-STEP GENERATED SOLUTION:
\begin{array}{cl} ax+3, &\text{if }x>2, \\
x-5 &\text{if } -2 \le x \le 2, \\
Solve this olympiad algebra problem step by step.
2x+b &\text{if } x <-2.
\end{array}
Problem:
\right.\]Find $a+b$ if the piecewise function is continuous (which means that its graph can be drawn without lifting your
The sequence of integers in the row of squares and in each of the two columns of squares form three distinct arithmetic se
FINE-TUNED MODEL STEP-BY-STEP GENERATED SOLUTION:
```